
SYSTEM EXPLAINER · V1

The App Factory

A semi-autonomous pipeline where AI agents build personal web apps overnight, and one person reviews the results in fifteen minutes over coffee.

What this is

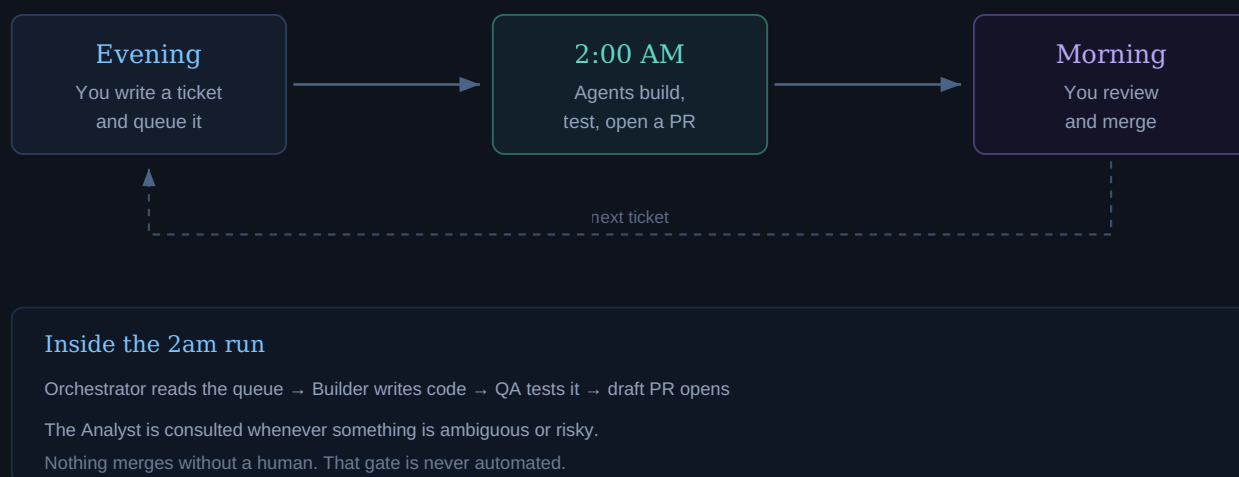
Most people use an AI coding assistant by sitting next to it. You open a session, describe what you want, watch it work, correct it, and close the session. The AI is a tool you operate.

This system is arranged differently. Work is described in advance, queued, and picked up by a scheduled agent that runs in the cloud at 2am while the laptop is shut. In the morning there is a pull request waiting, with a plain-English summary and a link to a working preview of the app. The owner reads a paragraph, taps the link, tries the feature, and either merges it or writes a sentence explaining what is wrong.

The design target is **ten to fifteen minutes of human involvement per run night**. Not because the human is lazy, but because anything more expensive than that stops happening by week three.

The interesting constraint is not capability. It is quota. Everything in this document exists because one Claude Pro subscription has to cover the overnight agents, the daytime chat, and the morning review — from a single shared pool.

The shape of it



The loop. The laptop is closed for the middle box.

Where everything runs

Piece	What it is
Claude Code Routines	Scheduled cloud agents on Anthropic's infrastructure. This is what fires at 2am. Billed against the Pro subscription, not a separate API bill.
GitHub	One repository per app. Holds the code, the ticket queue (as issues), the agent definitions, and the decision log.
Vercel	Hosting. Every pull request automatically gets its own live preview URL — this is what makes reviewing from a phone actually work.
Supabase	Database and auth, where an app needs them.
The apps	Progressive web apps. One codebase that installs to an iPhone home screen and runs in a laptop browser. No App Store, no Xcode.

Nothing runs on the laptop. Local overnight execution was considered and rejected: Macs sleep, scheduling is fiddly, and failures are silent. Everything the agent needs already lives in the GitHub repository.

The team

Four roles, but only three of them are separate agents. The fourth is the run itself.

Role	Job
Orchestrator	Not a separate agent — this <i>is</i> the scheduled run's top-level session. It reads the queue, picks tickets, dispatches the others, moves state, writes the decision log, and opens pull requests.
Analyst	Holds the product brief. Classifies every question and every intended action against the escalation tiers. Also does an evening pass over queued tickets, flagging ambiguity while the human is still awake to answer.
Builder	Writes the code. One ticket per invocation, on its own branch.
QA	Tests the work against the ticket's definition of done, item by item. On pass, writes the review packet the human actually reads.

They cannot talk to each other. Every exchange routes through the orchestrator. That sounds inefficient and is in fact deliberate: it keeps one agent responsible for state, and it means a subagent that goes wrong can only affect its own narrow output.

A role that was removed

An earlier design had a fifth agent, a Reviewer. It was cut because its function was almost entirely duplicated by QA's final pass plus the mandatory human merge gate — at a cost of roughly one-sixth of every ticket's token budget. Its one genuinely distinct output, the human-readable summary, moved to QA. It gets reinstated only if merged work starts shipping problems the remaining chain misses.

The safety mechanism

An agent working unsupervised at 3am cannot ask permission. There is no prompt, no approval dialog, nobody awake. So the question "what is this thing allowed to decide on its own?" has to be answered in advance, in writing, precisely.

Three tiers do that work.

Tier 1 — stop and wait for the human

Real money moving anywhere. Storing or collecting personal data. **Anything requiring a new account, sign-up, API key or credential — free or not.** Destructive operations on live data.

The ticket stops. The run continues with the next one.

Tier 2 — decide, proceed, flag it loudly

The test is one question: *would this be expensive to reverse after ten more tickets are built on top of it?* Data structure choices. Framework and major library choices. Committing to an outside service. These get logged as high-impact, and every entry must state **why**, not just what.

Tier 3 — decide, proceed, log a line

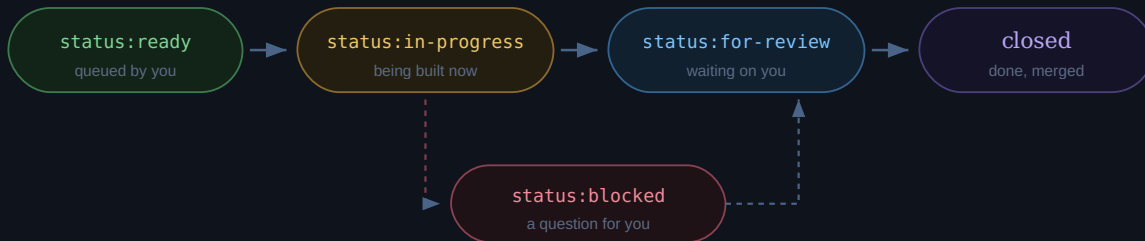
Everything else. Conventions, layout, formatting, sensible defaults.

Two structural rules sit above the tiers. First, **the tiers classify decisions, not just questions** — an agent that never asks anything can still quietly make a Tier 1 decision by adding a data field "as a sensible default." Second, **a Tier 1 stop blocks the ticket, never the run.** One risky ticket must not waste a whole night.

The subtle failure mode here is not a Tier 1 that fires too rarely. It is one that fires too often. Repeated false alarms pressure the owner into loosening the wording, and the looseness then leaks into the cases that mattered. When Tier 1 fires wrongly twice on the same pattern, the fix is to answer the question in the product brief — never to soften the tier.

How work is tracked

Each ticket is a GitHub issue. Its state lives in a single label:



Five states. "Done" is simply the issue being closed, which happens automatically when you merge.

This was originally a GitHub project board with columns. It isn't any more, and the reason is worth recording: **a scheduled cloud agent physically cannot reach GitHub Projects**. The API it would need is GraphQL-only, and the security proxy those runs sit behind blocks GraphQL entirely. This was established by running two diagnostic probes rather than by reading documentation. Labels do the same job, are reachable, and have one advantage the board never had — a blocked ticket sends a phone notification.

What the human actually reads

The whole safety model routes through one person glancing at something on a phone at 8am. If that something is a code diff, the realistic outcome by week three is rubber-stamping green ticks. So the artifact is specified rather than left to chance. Every pull request description contains exactly five things, written by QA:

1. **The preview URL, on the first line.** The review is *using the app*, not reading the diff.
2. **What changed, in plain language.** Three to six sentences. No file names — a file path is an invitation to go read the diff.
3. **High-impact decisions**, each with its *because*. Or the explicit word "None."
4. **What was tested, and what was not.** Every unverifiable item named individually.
5. **One line on what to look at specifically.**

Point four is the one that does the heavy lifting. QA has no phone, so anything requiring a real device comes back as CANNOT VERIFY and is named. An honest "I did not test this" is worth far more than a packet that implies coverage it doesn't have.

Deliberate omissions

A fresh reader — human or AI — tends to suggest the same handful of additions. Each was considered and rejected for a reason.

Rejected	Why
Multi-agent "teams"	Roughly three to four times the token consumption for the same task. Wrong economics on a Pro subscription.
Auto-merging pull requests	The manual gate costs about ten seconds and is the only thing preventing unreviewed code from accumulating.
Running the job on the laptop	Macs sleep. Scheduling is fiddly. Failures are silent.
Linting tickets inside the run	Defeats the point. The whole value of the lint pass is that the human is awake to answer.
Stacking multiple design skills	Two opinionated rulebooks produce two competing sets of instructions in the same context.
A separate messaging bot for dispatch	Machinery solving a problem the scheduler and the mobile app already solve.

Honest limitations

The bottleneck is specification, not intelligence

Bad runs almost always trace to a vague ticket rather than an agent failure. During the shakedown, two test tickets were written without first checking the codebase: one asked for work already done, the other assumed a screen that didn't exist. Both were caught — one by the lint pass, one by the Builder — but each cost a cycle. The person writing tickets is the limiting factor.

Review decay is the real risk

The failure mode isn't a dramatic bad merge. It's the slow slide into approving without looking. The tell is simple: if you can't answer "what did I merge yesterday?", the gate has already stopped working.

The platform is a research preview

Scheduled cloud agents are new. Run caps, trigger behaviour and network restrictions have all shifted during setup. Several assumptions in the original design turned out to be wrong when tested — which is why the system now prefers running a cheap diagnostic over trusting documentation.

Design distinctiveness needs grounding

Design skills move a model off the centre of the distribution, but they supply no picture of what *this* app should look like. That requires reference material chosen by a human. Skills alone produce generic-but-slightly-better.

What makes this system interesting is not that AI writes the code. It is that the arrangement makes an unsupervised agent's mistakes cheap, visible and reversible — a queue it cannot jump, a gate it cannot open, and a written record of every judgement it made while nobody was watching.